

Online Game Level Editor

ARTISAN WORLD CONNECTION SYSTEM

Presented by

M. Shiva Nandheswara Reddy

J. Kalyan Ram

02 / Problem Statement

THE PROBLEM

Modern gaming is saturated with generic, procedurally generated AI "noise." Automatic level generation lacks the intentional pacing, emotional arcs, and tactical logic required for high-tier game experiences.

OUR GOAL

Develop a robust C program utilizing **Graph Data Structures** that empowers human architects to build interconnected zones with 100% intent, supported by persistent binary storage.



Zone Connectivity



CRUD Logic



Graph Theory



File Persistence

03 / Real-World Applications



RPG Dungeon Maps

Connecting quest rooms and boss zones.



Shooter Arenas

Optimizing tactical flanking routes.



Open World Hubs

Managing travel between major cities.



Logic Puzzles

Trigger-based level progression.

KEY BENEFITS: Intentional difficulty spikes, lightning-fast zone traversal queries, and artisan-quality map narrative flow.

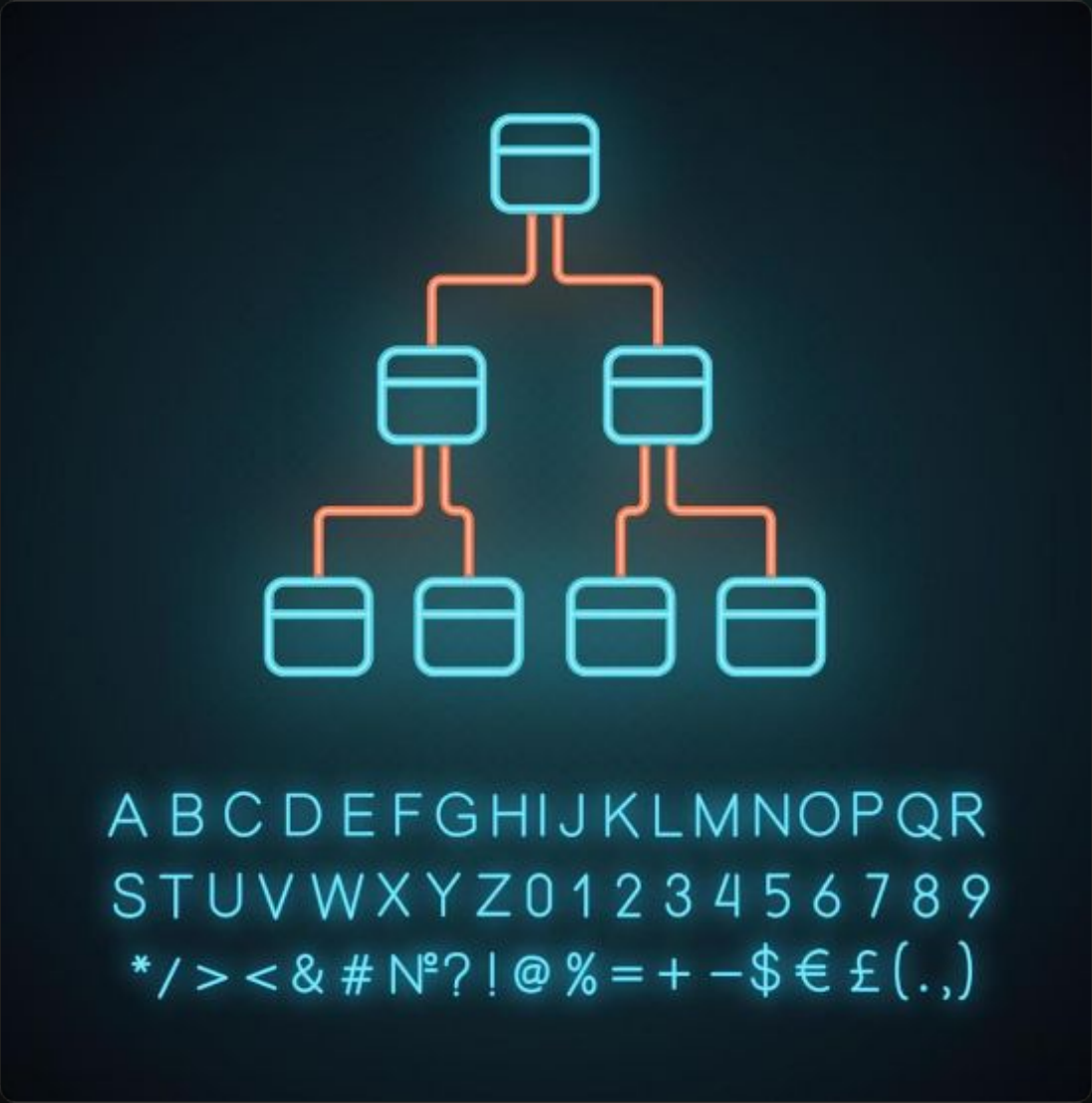
04 / Data Structure Used

Node: Individual Room, Zone, or Landmark.

Edge: Portal, Door, or Teleporter connection.

Weight: Difficulty level or Travel cost (Mana/Time).

Matrix Index	Zone A	Zone B
Zone A	0	15 (Weighted)
Zone B	15	0



05 / Core Data Structure

```
typedef struct {
```

```
    char zones[MAX][50]; // Zone identifiers
```

```
    int graph[MAX][MAX]; // Adjacency Matrix
```

```
    int zoneCount;        // Current registered zones
```

```
} LevelGraph;
```

zones[][]

Authored names array.

graph[][]

Logic matrix weights.

zoneCount

Active nodes registry.

06 / System Design

Add Zone

Register new spatial data.

Add Portal

Connect zones in matrix.

Delete Logic

Prune unreachable paths.

Update Flow

Modify traversal weights.

Search Path

Query matrix cell values.

Show Graph

Render ASCII connectivity.

APPLICATION FLOW: Start → Load (level.dat) → Menu → Run Logic → Save → Exit

07 / Algorithm

01 Load Data: Deserialize level.dat into binary struct.

02 Display Menu: User-driven console interface interaction.

03 Execute Op: Real-time CRUD on graph adjacency nodes.

04 Save Data: Serialize updated struct back to disk.

➡ Add Zone → Store in String Array

➡ Add Portal → Update Matrix [i][j]

➡ Book Traversal → Validate Edge existence

08 / Sample Output

=== GAME LEVEL EDITOR ===

1. Add Zone 2. Link Portal

3. Delete Path 4. Update Cost

5. Query Connection 6. Map Audit

Enter choice: 5

Source Zone: DUNGEON_ENTRANCE

Target Zone: BOSS_CHAMBER

Connection Logic: EXISTS | Difficulty: 45pts

09 / Program

Implementation

Adjacency matrix manages all interconnectivity and pacing weights.

 **Memory-Efficient:** Custom structs ensure zone names and logic are tied together.

 **Binary File I/O:** Persistence via level.dat—all artisan maps survive program exit.

 **CRUD Core:** Fully functional system for map architectural refinement.

```
src > routes > Offers > layout > JS OffersBreadCrumbs.js > [0]mapStateToGrainSort
129         id: grainTypeId,
130         } = {},
131     },
132 },
133 grains: { byGrainTypeId },
134 } = state;
135 if (!!grainUri, !grainTypeId) return;
136 return (byGrainTypeId[grainTypeId].byUri[grainUri] || {}).id;
137 };
138
139 const mapStateToProps = (state, { offerId }) => {
140     return {
141         filter: state.offers.filter,
142         offer: (state.offers.details.byId[offerId] || state.offers.details.defaults).da
143         grainTypeId: (state.offers.filter.grainType || {}).id, // NOTE: for fetching gr
144         grainId: (state.offers.filter.grain.length === 1) ? getGrainId(state) : undefin
145     };
146 };
147
148 const mapStateToRegion = {
149     mapToProps: ({ byUri, byId }, { offer, filter }) => {
150         const region = offer
151             ? byId[offer.region_id]
152             : byUri[filter.region_uri];
153         return {
154             region,
155             regionId: region ? region.id : undefined
156         }
157     },
158     shouldFetch: (state, { offer, filter }) => offer ? !!offer.region_id : !!filter.regi
159 };
160
161 const mapStateToFitu = {
```

1 asset
Entrypoint undefined = search.html
[./node_modules/html-webpack-plugin/lib/loader.js?./src/routes/Offers/layout/JS/OffersBreadCrumbs.js]

10 / Conclusion

Artisan Worlds

Verified zone connectivity for human-made levels.

Graph Logic

Matrix representation for optimized traversal logic.

Binary Persistence

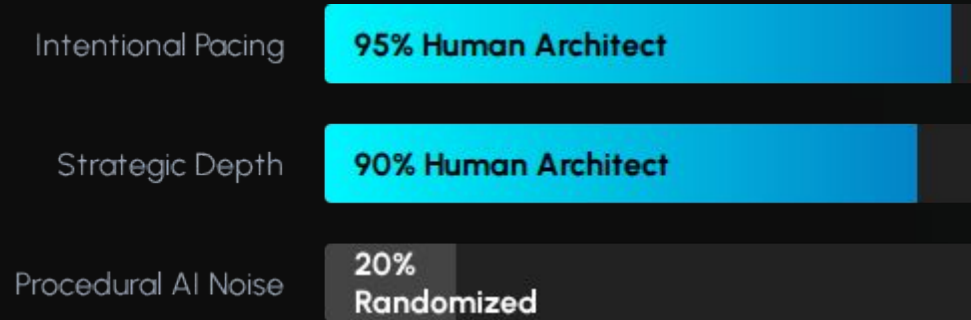
Robust file handling for level data integrity.

User Interaction

Streamlined menu-driven console system.

FUTURE IMPROVEMENTS: Integration of Dijkstra's Algorithm for dynamic pathfinding and a GUI overlay for spatial visualization.

11 / Human vs AI Analysis



Thank

Questions & Discussion Welcome
You!

M. Shiva Nandheswara Reddy

J. Kalyan Ram

C Programming | Graph Data Structures | Level Systems Design

Image Sources



https://static.vecteezy.com/system/resources/previews/004/556/159/non_2x/tree-diagram-neon-light-icon-hierarchical-system-node-link-diagram-tree-structure-sequence-and-submission-glowing-sign-with-alphabet-numbers-and-symbols-isolated-illustration-vector.jpg

Source: www.vecteezy.com



<https://cdn.mos.cms.futurecdn.net/v2/t:0,l:437,cw:1125,ch:1125,q:80,w:1125/joKDb6aZb5tpwFVT4QxbQc.jpg>

Source: www.itpro.com



<https://preview.redd.it/procedural-vs-hand-made-levels-which-is-better-for-our-v0-csg8zcumu3hg1.png?width=640&crop=smart&auto=webp&s=0e50e687434a591eaca4cab553ea0a43457b1053>

Source: www.reddit.com